

تکلیف چهارم درس امنیت شبکه

نویسنده: محمد امانلو

شماره دانشجویی: ۸۱۰۱۰۰۰۸۴

استاد: دکتر مهسا سعیدی

سوال اول

در این شکل، کارمند می خواهد با استفاده از ESP Tunnel Mode برای دورکاری به سرور داخلی شرکت وصل شود. سرور داخل شبکه شرکت است و آدرسش 192.168.1.2 است. از آن طرف Security Gateway هم یک آدرس بیرونی 8.8.8.8 دارد و یک آدرس داخلی 192.168.1.1.

الف

اینجا SA بین کارمند و خود سرور ساخته نشده، چون سرور داخل شبکه محلی شرکت قرار دارد و آدرس 192.168.1.2 یک آدرس داخلی است. یعنی این سرور مستقیم روی اینترنت قابل دسترسی نیست و قرار نیست خودش endpoint اصلی IPsec باشد.

پس در عمل، کارمند نمی تواند مستقیم با خود سرور یک ارتباط Host-to-Host IPsec برزند. کاری که انجام میشه اینه که کارمند SA را با Security Gateway می سازد، چون Gateway همان نقطه ای است که از اینترنت دیده می شود. بعد بسته های رمز شده تا Gateway میرسن و از آنجا به بعد داخل شبکه داخلی برای سرور فرستاده می شوند.

کلا Tunnel Mode بیشتر برای حالتی مناسب است که endpoint امنیتی با مقصد واقعی یکی نیست، مثل همین ارتباط Host-to-Gateway یا مثلاً Gateway-to-Gateway. ولی برای ارتباط مستقیم بین دو میزبان انتهایی، معمولاً Transport Mode کافی است، چون دیگر لازم نیست یک Outer IP header اضافه کنیم و آدرس های داخلی را پشت یک تونل قایم کنیم.

ب

اگر کارمند بخواهد هم ESP داشته باشد و هم AH، بهتره اول ESP اعمال شود و بعد AH روی خروجی آن بیاید. دلیلش این است که ESP کار رمزنگاری و محرمانگی داده اصلی را انجام می دهد، بعد AH می تواند روی بسته بیرونی Authentication و کنترل یکپارچگی انجام دهد.

با این ترتیب، وقتی بسته به سمت مقابل میرسه، گیرنده اول AH را چک می کند. اگر بسته وسط راه دستکاری شده باشد یا از طرف فرستنده معتبر نباشد، همان اول drop می شود و دیگر سیستم بیخودی وارد ESP decryption نمی شود. این موضوع مخصوصا از نظر کم کردن پردازش اضافه و جلوگیری از بعضی حمله های شبیه DoS هم مفید است.

از طرف دیگر، وقتی AH بیرون ESP قرار بگیرد، می تواند خود ESP header و قسمت های تغییر ناپذیر Outer IP header را هم پوشش دهد. پس ترتیب مناسب تر اینطوری است:

ابتدا Inner ESP و بعد Outer AH

ج

نه، لازم نیست روی خود سرور هم IPSec پیاده سازی شود. در این سناریو endpoint تونل IPSec همان Security Gateway است. یعنی بسته از اینترنت به Gateway میرسه، همان جا IPSec header ها بررسی و حذف می شوند و محتوای بسته هم decrypt می شود.

بعد از این مرحله، Gateway بسته اصلی را مثل یک IP packet معمولی داخل شبکه محلی برای سرور 192.168.1.2 میفرسته. پس سرور لازم نیست چیزی از IPSec بداند یا خودش IPSec را پشتیبانی کند. از دید سرور، یک بسته عادی از شبکه داخلی به دستش رسیده است.

برای بسته ای که از سمت کارمند به سمت سرور می رود، در Tunnel Mode دو تا IP header داریم:

- **Inner IP header:** این همان بسته اصلی است که سیستم کارمند تولید کرده و مقصد واقعی آن سرور داخلی است. پس آدرس ها اینطوری هستند:

Source IP = 1.1.1.1

Destination IP = 192.168.1.2

این header همراه با داده اصلی داخل ESP payload قرار می گیرد، پس رمز شده است و کسی در مسیر اینترنت نمی تواند مقصد واقعی بسته را از روی آن بخواند.

- **Outer IP header:** این header برای این اضافه می شود که بسته بتواند از اینترنت عبور کند و به Security Gateway برسد. پس آدرس های آن اینطوری است:

Source IP = 1.1.1.1

Destination IP = 8.8.8.8

این بخش رمز نمی شود، چون روترهای اینترنت باید بتوانند آدرس مقصد بیرونی را ببینند و بسته را تا Gateway مسیریابی کنند.

سوال دوم

وقتی دو کاربر هر دو پشت NAT باشند و بخواهند با ESP Transport Mode ارتباط بگیرند، مشکل اصلی این است که PAT برای کار کردن به شماره پورت های TCP یا UDP نیاز دارد. یعنی NAT باید بتواند پورت مبدا را ببیند و با تغییر آن، چند ارتباط مختلف را از هم جدا کند. اما در ESP Transport Mode بخش مربوط به TCP/UDP header داخل داده رمز شده قرار می گیرد. از طرفی خود ESP هم پورت ندارد، چون مثل TCP و UDP پروتکل لایه انتقال نیست. برای همین PAT دیگر نمی تواند پورت ها را بخواند یا تغییر بدهد و عملاً ارتباط درست از پشت NAT رد نمی شود. مشکل دوم مربوط به Checksum است. در TCP و UDP مقدار Checksum با توجه به آدرس های IP مبدا و مقصد هم حساب می شود. وقتی NAT آدرس IP را عوض می کند، باید Checksum را هم دوباره حساب کند. ولی چون TCP/UDP header داخل ESP رمز شده، NAT به آن دسترسی ندارد و نمی تواند Checksum را اصلاح کند. در نتیجه ممکن است بسته در مقصد نامعتبر تشخیص داده شود و دور انداخته شود.

راه حل این مشکل استفاده از IPsec NAT Traversal یا همان NAT-T است. در این روش، بسته های ESP داخل یک بسته UDP کپسوله می شوند. معمولاً از پورت 4500 UDP استفاده می شود. در نتیجه NAT به جای اینکه با یک بسته ESP خام روبرو شود، یک بسته UDP معمولی می بیند و می تواند روی UDP header بیرونی عملیات PAT را انجام دهد.

ساختار کلی بسته تقریباً اینطوری می شود:

Outer IP header | UDP header | ESP header | Encrypted TCP/UDP data | ESP trailer

با این کار، PAT فقط پورت موجود در UDP header بیرونی را تغییر می دهد و دیگر لازم نیست به محتوای رمز شده ESP دست بزند. پس هم مشکل پنهان بودن پورت های اصلی حل می شود، هم مشکل Checksum کمتر می شود، چون Checksum داخلی دست نخورده باقی می ماند و فقط بخش بیرونی توسط NAT تغییر می کند. در مقصد هم UDP

header بیرونی برداشته می شود و بسته ESP اصلی مثل حالت عادی پردازش می شود.

سوال سوم

در این بخش فایل ssl3.0.cap را در Wireshark باز کردم و Message های مربوط به اولین Session از نوع SSL3 را بررسی کردم. برای جدا کردن Message های اصلی هم از فیلترهای مربوط به tls.handshake.type استفاده شد.

الف

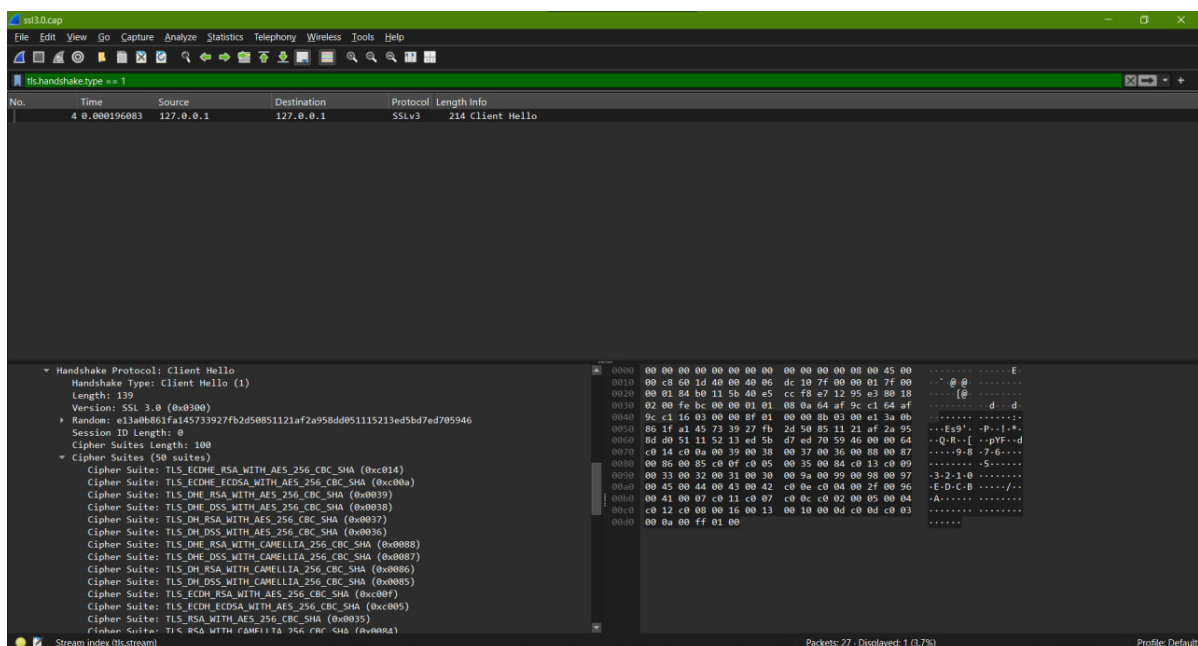
در اولین Session، Packet شماره ۴ مربوط به Client Hello است. داخل این Packet، کلاینت لیست Cipher Suite های پیشنهادی خودش را در بخش Cipher Suites فرستاده است. تعداد این گزینه ها برابر ۵۰ تا بود. ضعیف ترین گزینه ای که در این لیست دیده می شود، این مورد است:

TLS_RSA_WITH_RC4_128_MD5

دلیل ضعیف بودنش اینه که از RC4 برای Encryption استفاده می کند و RC4 عملاً الگوریتم امنی حساب نمی شود، چون در برابر تحلیل های آماری ضعیف دارد. از طرف دیگر، MD5 هم برای بحث Integrity و Hash گزینه قابل اعتمادی نیست و مدت هاست شکسته شده. برای همین این ترکیب جزو ضعیف ترین گزینه های لیست است. قوی ترین گزینه مناسب در این Session هم این مورد است:

\texttt{TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA}

اینجا ECDHE باعث می شود Key Exchange خاصیت Forward Secrecy داشته باشد، یعنی اگر بعداً Private Key سرور لو برود، Session های قبلی راحت قابل بازیابی نیستند. از طرف دیگر، AES-256 هم از نظر Symmetric Encryption گزینه قوی تری نسبت به الگوریتم هایی مثل RC4 و 3DES است. پس بین گزینه های قابل استفاده در این Session، این ترکیب انتخاب قوی تری حساب می شود.



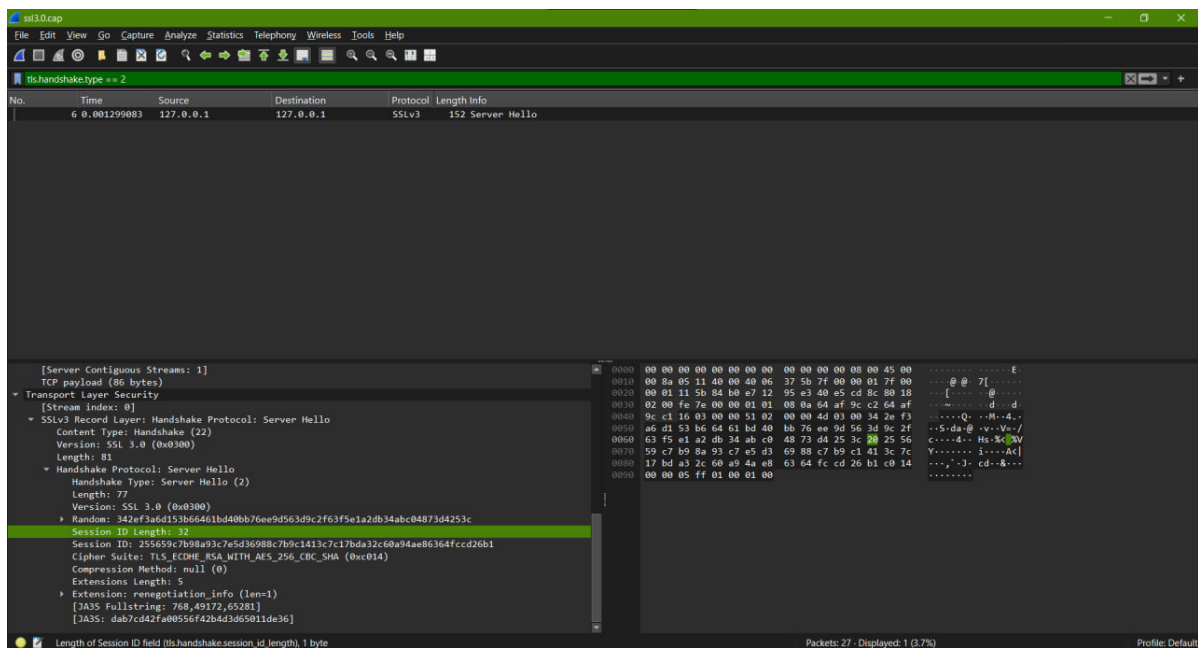
شکل ۱: بسته شماره ۴، Client Hello و لیست Cipher Suites پیشنهادی کلاینت

ب

برای جواب این قسمت، Packet شماره ۶ که Server Hello است را بررسی کردم. در این Message، سرور از بین گزینه هایی که کلاینت پیشنهاد داده بود، Cipher Suite زیر را انتخاب کرده است:

TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA

این انتخاب یعنی سرور یکی از گزینه های قوی لیست کلاینت را قبول کرده است. این گزینه هم از ECDHE برای Key Exchange استفاده می کند، هم از AES-256 برای Symmetric Encryption، و Signature سرور هم مبتنی بر RSA است. پس انتخاب سرور نسبت به گزینه های ضعیف تر مثل RC4 یا MD5 انتخاب امن تری بوده.



شکل ۲: بسته شماره ۶، Server Hello و Cipher Suite انتخاب شده توسط سرور

پ

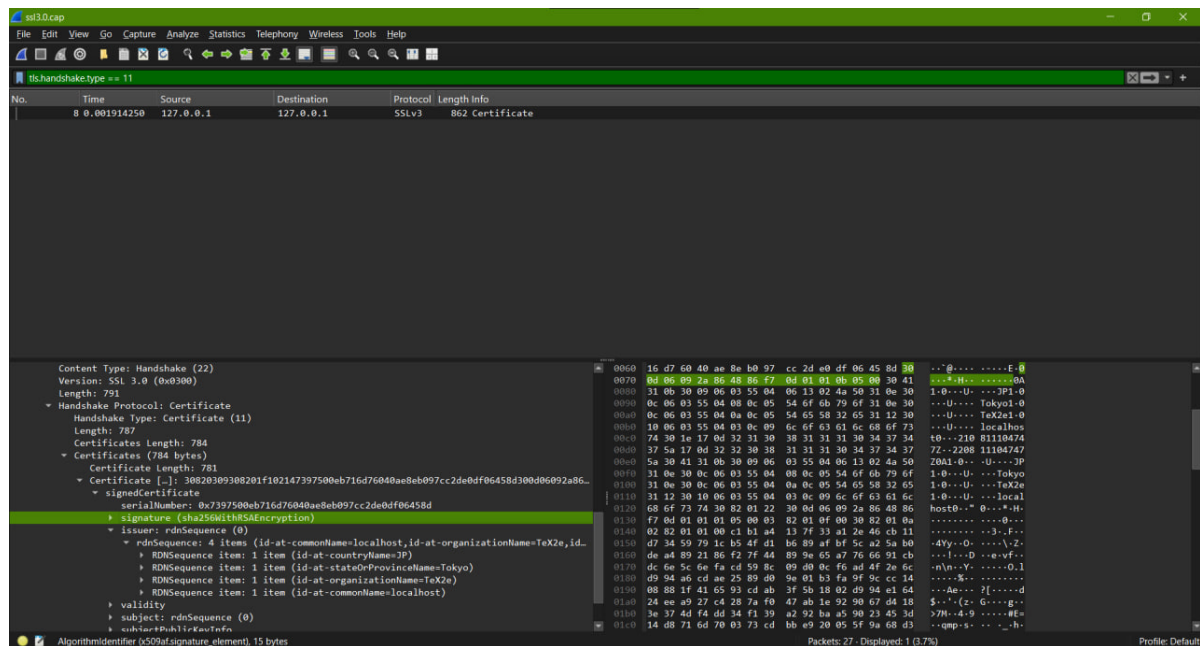
برای پیدا کردن شرکت صادر کننده Certificate سرور، Packet شماره ۸ را بررسی کردم که Certificate داخل آن قرار دارد. در بخش issuer، مقدارهای مربوط به صادر کننده Certificate دیده می شود. طبق خروجی Wireshark، مقدار organizationName برابر است با:

TeX2e

همچنین مقدار commonName برابر است با:

localhost

پس شرکت یا سازمان صادر کننده Certificate سرور همان TeX2e است. مقدار localhost هم نشان می دهد این Certificate بیشتر برای محیط تست یا اجرای محلی ساخته شده، نه برای یک دامنه واقعی روی اینترنت.



شکل ۳: بسته شماره ۸، بخش Certificate و اطلاعات issuer

ج

برای بررسی TLS 1.2 اول از بخش Conversations تعداد TCP Stream ها را چک کردم. طبق خروجی فایل، فقط یک TCP Stream با ID = 0 وجود دارد و تعداد کل Packet های آن هم ۲۰ تاست. پس بر خلاف چیزی که در متن سوال گفته شده، در این فایل عملاً دو Connection جدا دیده نمی شود و امکان مقایسه واقعی دو Connection با هم وجود ندارد.

در همین Connection موجود، ترتیب Handshake Message ها به این شکل است:

Packet 4: Client Hello

Packet 6: Server Hello, Certificate, Server Key Exchange, Server Hello Done

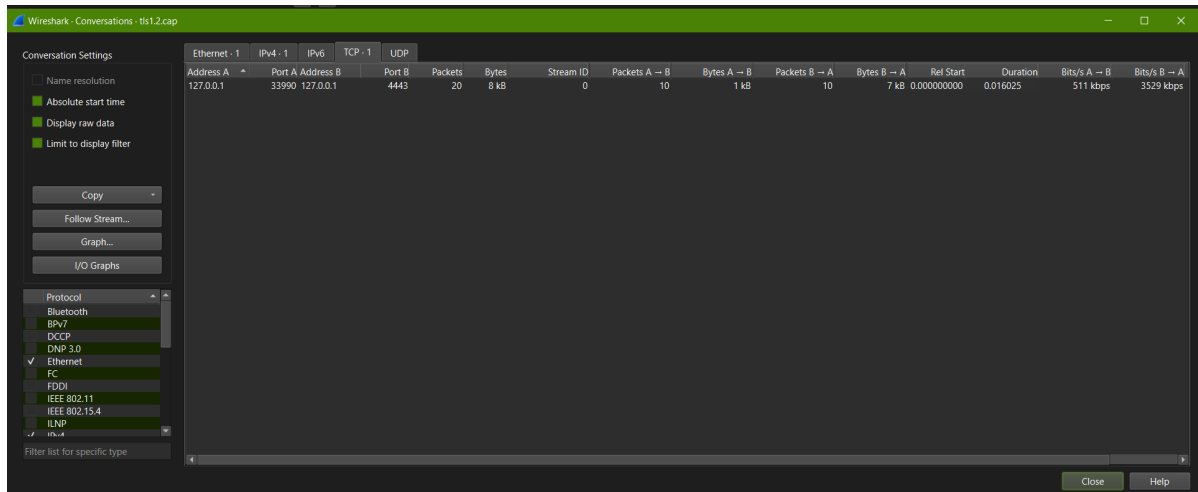
Packet 8: Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message

Packet 10: New Session Ticket, Change Cipher Spec, Encrypted Handshake Message

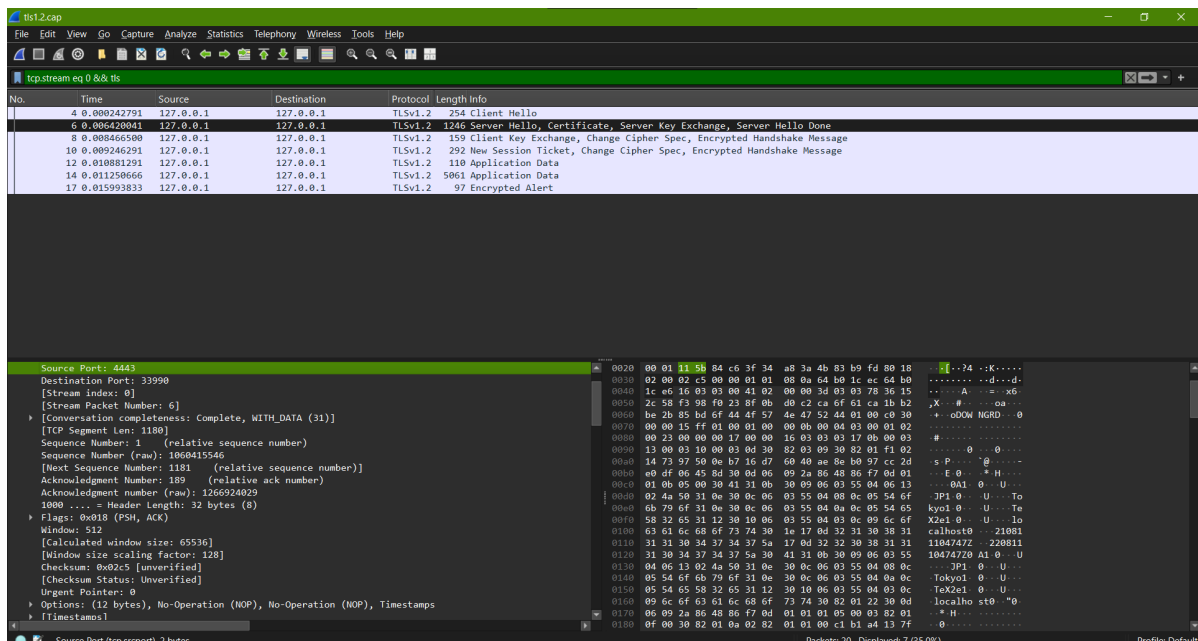
تا قبل از Change Cipher Spec، پیام هایی مثل Client Hello، Server Hello، Certificate، Server Key Exchange، Client Key Exchange و Server Hello Done به صورت Cleartext دیده می شوند. یعنی خود Wireshark محتوای آن ها را باز می کند و می توان فیلدهایشان را دید.

اما بعد از Change Cipher Spec، پیام دیگر به صورت عادی دیده نمی شود و در Wireshark با عنوان Encrypted Handshake Message نمایش داده می شود. پس از اینجا به بعد، پیام های اصلی Handshake وارد حالت Encrypted می شوند. همین موضوع در Packet شماره ۸ برای سمت کلاینت و در Packet شماره ۱۰ برای سمت سرور دیده می شود.

در نتیجه، چون در فایل فقط یک TCP Stream وجود دارد، مقایسه کامل بین دو Connection ممکن نیست. ولی در همین Connection می شود دید که بخش اول Handshake آشکار است و بعد از Change Cipher Spec پیام های حساس به صورت Encrypted منتقل می شوند.



شکل ۴: نمای Conversations در فایل tls1.2.cap که فقط یک TCP Stream با Stream ID = 0 را نشان می دهد



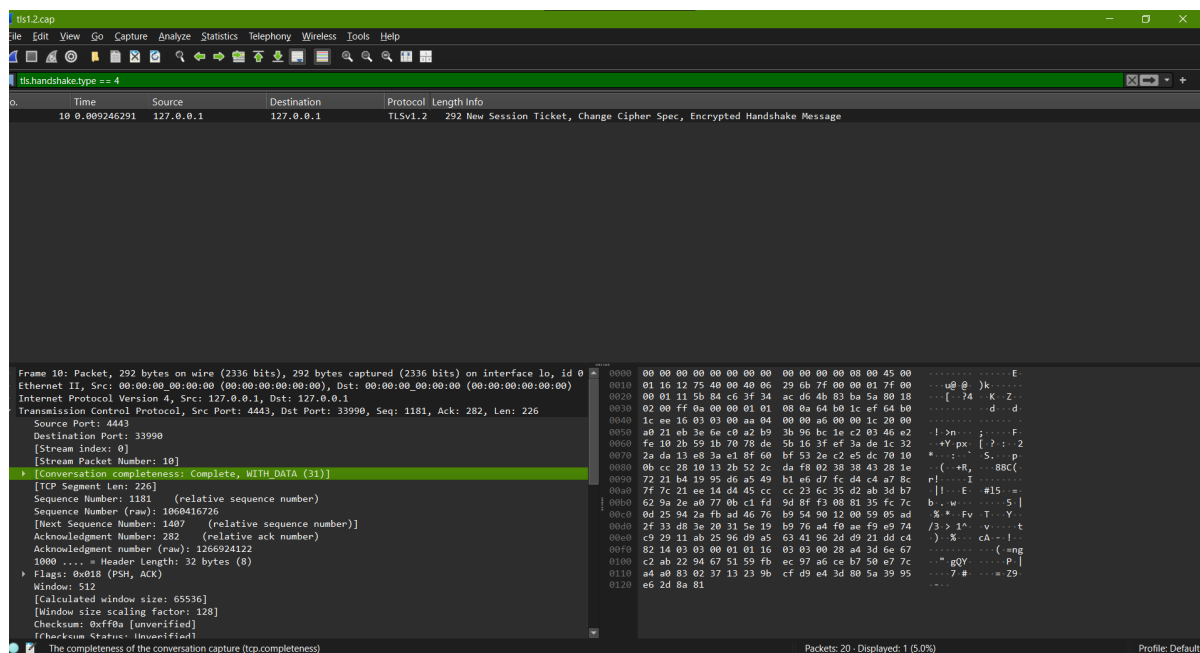
شکل ۵: ترتیب Handshake Message ها در تنها TCP Stream موجود در فایل TLS 1.2

چ

برای بررسی کوتاه شدن Connection دوم، دنبال سازوکار Session Resumption گشتم. در فایل فعلی فقط یک Connection کامل داریم، پس نمی شود عملاً نشان داد که Connection دوم کوتاه تر شده است. با این حال، در همین Connection یک نشانه مهم وجود دارد: در Packet شماره ۱۰ پیام New Session Ticket دیده می شود.

وجود New Session Ticket یعنی سرور بعد از پایان بخش اصلی Handshake، یک Ticket به کلاینت می دهد تا کلاینت بتواند برای Connection های بعدی از آن استفاده کند. اگر بعداً کلاینت دوباره بخواند به همین سرور وصل شود، لازم نیست کل Full Handshake از اول تکرار شود. کلاینت می تواند همان Session Ticket را در Client Hello بعدی ارائه بدهد و سرور در صورت معتبر بودن آن، Session قبلی را resume کند. این کار باعث می شود پیام هایی مثل Certificate، Server Key Exchange و بخشی از مراحل سنگین Key Exchange دوباره تکرار نشوند. در نتیجه تعداد رفت و برگشت ها کمتر می شود و کلاینت سریع تر می تواند به مرحله دریافت Application Data برسد. برای صفحات وب هم این موضوع مهم است، چون کم شدن زمان Handshake باعث می شود ارتباط زودتر آماده شود و بارگذاری صفحه سریع تر انجام بگیرد.

پس در این فایل، خود Connection دوم دیده نمی شود، ولی وجود New Session Ticket در Packet شماره ۱۰ نشان می دهد که سرور امکان Session Resumption را برای Connection های بعدی فراهم کرده است.



شکل ۶: Packet شماره ۱۰ شامل New Session Ticket، Change Cipher Spec و Encrypted Handshake Message

ح

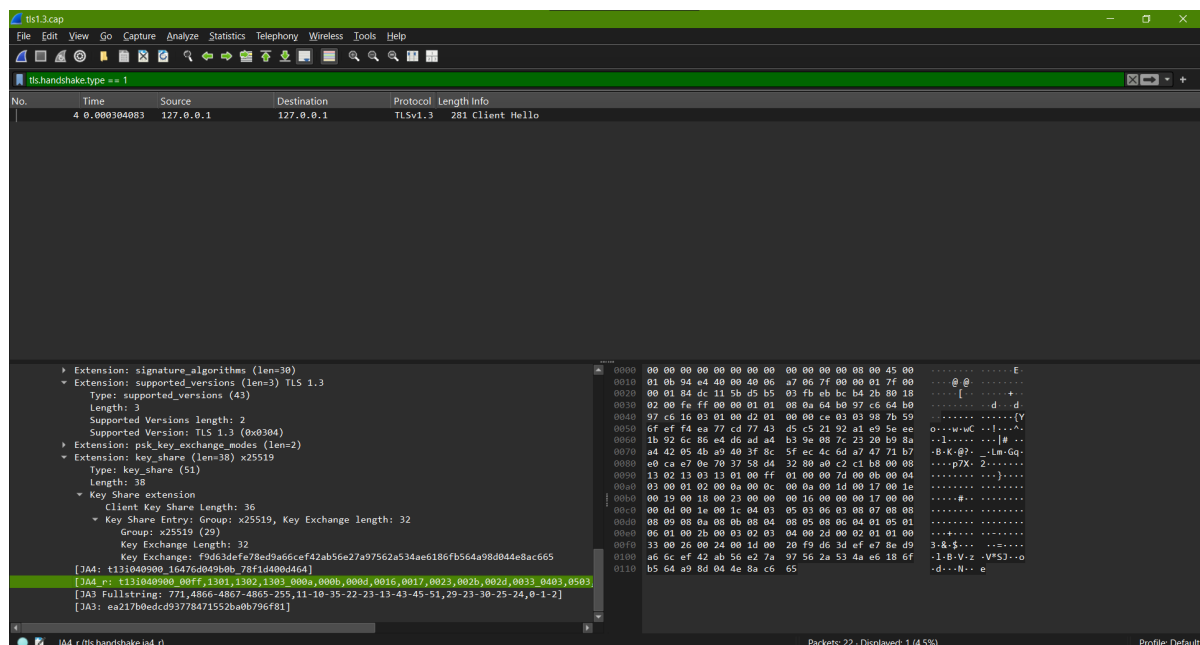
در فایل tls1.3.cap، Packet شماره ۴ مربوط به Client Hello است. در این Packet مقدار supported versions برابر TLS 1.3 دیده می شود. همچنین داخل همین Client Hello، یک extension به نام key share وجود دارد که مقدار Group آن برابر x25519 است.

Client Hello Packet = 4

supported versions = TLS 1.3

key share group = x25519

دلیل اینکه در TLS 1.3 پیام جداگانه ای به اسم Client Key Exchange نداریم، همین است که اطلاعات لازم برای Key Exchange از همان اول داخل Client Hello و در بخش key share فرستاده می شود. یعنی کلاینت منتظر یک پیام جدا برای ارسال کلید نمی ماند و از همان ابتدا Key Share خودش را می فرستد. این کار باعث می شود Handshake در TLS 1.3 کوتاه تر و سریع تر از نسخه های قبلی شود.



شکل ۷: Packet شماره ۴، Client Hello شامل supported versions = TLS 1.3 و key share با گروه x25519

Packet 8 Info = Change Cipher Spec, Application Data

در TLS 1.3، پیام Change Cipher Spec مثل TLS 1.2 نقش اصلی در عوض شدن حالت رمزنگاری ندارد. این پیام بیشتر برای سازگاری با تجهیزات قدیمی شبکه یا همان Middlebox Compatibility Mode فرستاده می شود. بعضی تجهیزات قدیمی انتظار دارند در مسیر Handshake یک پیام Change Cipher Spec ببینند، برای همین در TLS 1.3 هم این پیام به شکل ظاهری نگه داشته شده است.

در مورد اینکه کلاینت چگونه می‌تواند به سرور متصل شود، نکته اینجاست که کلاینت از قبل مقدار Client Hello را فرستاده و در Server Hello هم مقدار key share سرور دریافت شده است. با همین دو مقدار، دو طرف می‌توانند کلیدهای اولیه لازم برای ادامه Handshake را حساب کنند. برای همین بعد از Server Hello، بخش‌های بعدی مثل Application Data به صورت رمز شده دیده می‌شوند.



در TLS 1.3 وضعیت با TLS 1.2 فرق دارد. در TLS 1.2 معمولاً Certificate سرور به صورت واضح در Handshake دیده می شود، ولی در این فایل وقتی فیلتر `tls.handshake.type == 11` را زدیم، هیچ Packet ای نمایش داده نشد. یعنی بدون رمزگشایی، Certificate سرور قابل دیدن نیست.

دلیلش این است که در TLS 1.3 بعد از Server Hello، بیشتر پیام های Handshake به صورت رمز شده داخل Application Data نمایش داده می شوند. برای همین Wireshark بدون داشتن secret ها نمی تواند پیام هایی مثل Certificate را باز کند.

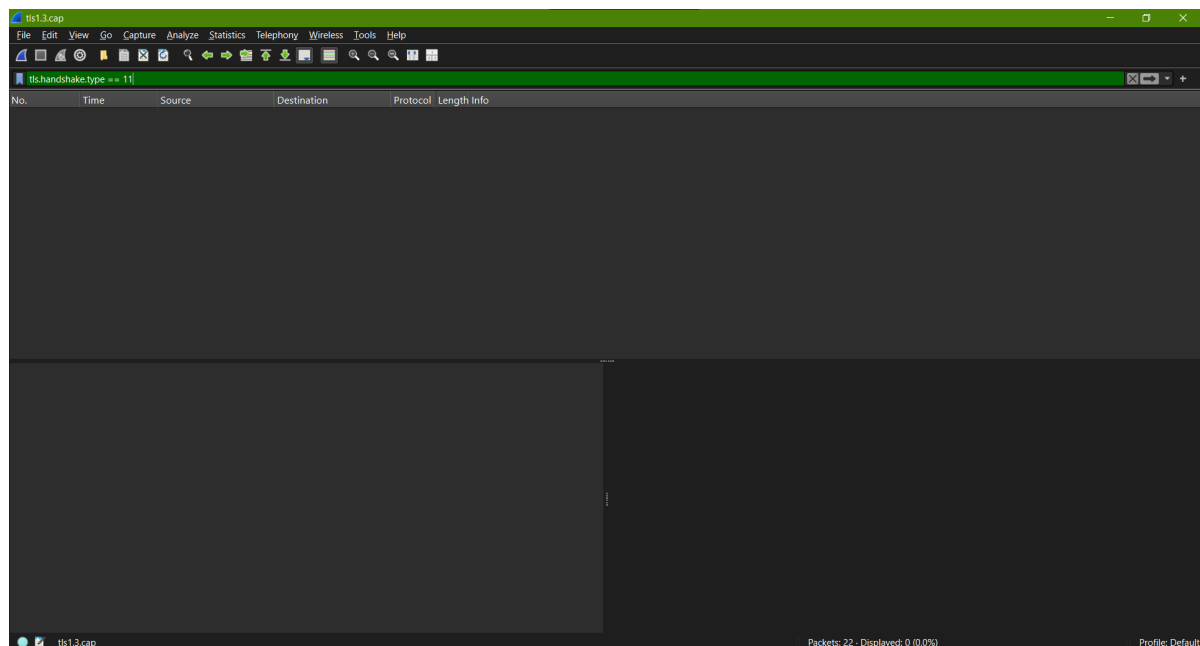
اما مقدار Session ID در Server Hello قابل مشاهده است. در Packet شماره ۶، فیلد Session ID Length برابر ۳۲ است و مقدار Session ID هم دیده می شود:

Server Hello Packet = 6

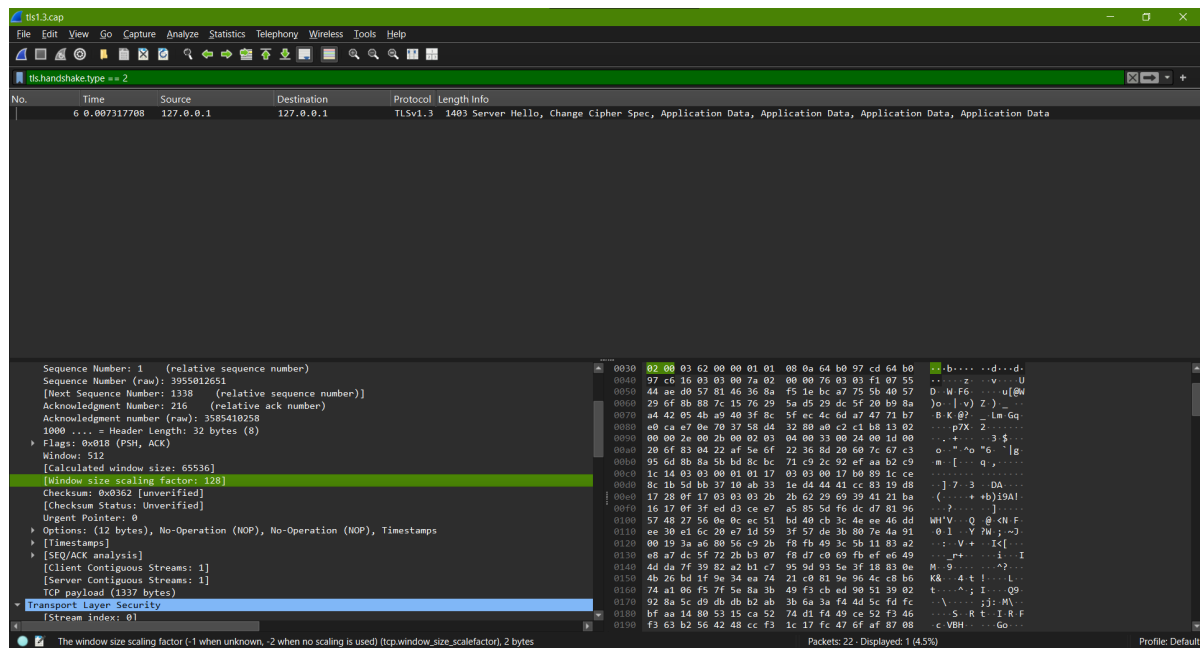
Session ID Length = 32

Session ID = b98aa442054ba9403f8c5fec4c6da74771b7e0cae70e703758d43280a0c2c1b8

البته این مقدار در TLS 1.3 بیشتر نقش legacy session id یا legacy session id echo دارد و مثل Session ID واقعی در TLS 1.2 برای Session Resumption استفاده نمی شود.



شکل ۹: عدم نمایش Certificate بدون رمزگشایی با فیلتر `tls.handshake.type == 11`



شکل ۱۰: Packet شماره ۶، Server Hello و مقدار Session ID در TLS 1.3

د

برای این قسمت باید مقادیر محرمانه یا فایل secret log در Wireshark وارد شود تا پیام های رمز شده TLS 1.3 قابل خواندن شوند. چون در این بررسی فایل secret در دسترس نبود، امکان رمزگشایی کامل پیام ها و استخراج دقیق New Session Ticket یا PSK identity وجود نداشت.

در حالت کلی، اگر secret ها وارد شوند، بعد از رمزگشایی باید پیام های مربوط به New Session Ticket دیده شوند. سرور این Ticket را به کلاینت می دهد تا کلاینت بتواند در اتصال های بعدی از آن به عنوان PSK identity استفاده کند و Session را ادامه بدهد. بنابراین مقدار استفاده شده برای ادامه Session در TLS 1.3 معمولاً از مسیر New Session Ticket و بعد pre shared key در Client Hello بعدی مشخص می شود. ولی چون secret log موجود نبود، من این بخش را فقط در حد قابل مشاهده بودن از روی فایل بررسی کردم و مقدار دقیق Ticket یا PSK identity را گزارش نکردم.